0.1 Introduction

The Input Module returns a validated graph object that carries the network topology together with its layered iteration sets and ancestry closures. All analysis modules in the framework consume this object directly as is.

The Network Decomposition Module, when invoked, extends the graph object with the network's decomposition structure. Like input processing, this is a network pre-processing step whose objective is to identify all instances of a specified subgraph pattern in the input DAG network.

0.1.1 Decomposition in Complex Networks

Real infrastructure networks are rarely trees. Their components connect to and depend on many others at once, and the interactions between them overlap rather than branch cleanly [1, 2]. Redundancy strategies are the obvious case: a robust network deliberately supplies more than one route to the same goal, so paths diverge and reconverge by design. Process networks and hierarchical flows behave the same way, with a component feeding several downstream dependencies while itself depending on shared upstream outputs. As the system grows, overlapping paths accumulate and the connectivity web thickens.

Graph-based analysis methods almost always do some form of decomposition, and for large networks with complicated connectivity it is a practical necessity [3]. Broadly, decomposition breaks a network into substructures that are easier to handle than the whole, and methods differ in what they do with those substructures. Some strategies aim to reduce: a substructure is replaced by a single equivalent element, so the network shrinks while the analysis stays exact. Other decomposition methods work by partition: the network is divided into units of work sized for the solver, and the pieces are processed in turn. Beyond either, decomposition can give a more compact representation of the network, storing a repeated substructure once rather than every time it appears, without losing the overall structure. Depending on how the decomposed units are defined, they can also support localised investigation, such as confining a sensitivity study to one part of the system without needing to run analysis against the full network.

There is no universal correct unit of decomposition; a decomposition method defines its own target substructure and partitions the network in those terms. In the classic series–parallel reduction, the reliability formula replaces each series or parallel pair with a single equivalent component and the network shrinks pair by pair [4]. The units here are the series and parallel pairs themselves. When the network is not series–parallel, that reducible structure is missing, and the factoring approach manufactures it: conditioning on a single component splits the problem into the component-working and component-failed cases, and the method recurses on each until it becomes reducible [5–7]. Polygon-to-chain reductions extend the same idea, widening what can be collapsed in one step

beyond the plain series and parallel pairs [8].

In dynamic programming the unit is the tree: a tree decomposition arranges the network into small overlapping bags of nodes connected in a tree, each interacting with the rest of the graph only through its neighbours, which is exactly the subproblem structure a dynamic program can process in a single sweep [9]. Working over bags rather than the whole graph is what makes exact reliability computation feasible on networks as large as metropolitan transit systems [10].

The framework's default decomposition unit is the *diamond*: a fork–join subgraph where paths in the network fan out from a common node and fan back in at a node further downstream. IPA's decomposition is partition-forward, that is, each identified diamond is returned as a localised graph object of the same form as the full network, and can be handed to any analysis module in the framework on its own. Secondarily, a diamond can serve as a reductive unit, since a solved diamond can stand in for its subgraph as a single equivalent component.

0.2 Preliminaries and Definitions

Identification works directly on the unified graph object of Chapter 3. From it the module reads the DAG $G = (V, E)$ and the iteration sets $L = \{L_1, \ldots, L_k\}$, whose validity is the Convergence Guarantee proved there. It also reads the ancestry closures, where the ancestor closure $\mathrm{Anc}(v)$ is self-inclusive by construction so that $v \in \mathrm{Anc}(v)$ for every node, and the fork and join classifications, a fork having fan-out above one and a join fan-in above one. The node value map travels with the object, but identification reads it in one place only, the exclusion rule of Section 0.4.7. The edge value map plays no part at all and is accepted only so the module's signature matches the rest of the framework.

Definition 1 (Topological index)**.** *Given the iteration sets L, define* $\mathrm{topi} : V \to \{1, \ldots, |V|\}$ *by numbering nodes in increasing order across layers and in sorted node order within each layer. Every node receives a distinct integer, so* topi *is a strict total order on V refining the layered order.*

Definition 2 (Conditioning context)**.** *A conditioning context $E \subseteq V$ is a set of nodes treated as fixed while assessing the relatedness of their descendants. Identification starts from $E = \varnothing$ and extends the context one node at a time as it recurses.*

Definition 3 (Unconditioned influence set)**.** *For a node p and context E,*

$$\mathrm{infl}(p, E) \;=\; \mathrm{Anc}(p) \setminus E.$$

Two parents p, q of a join are structurally correlated under E *when* $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) \neq \varnothing$*, that is, when some node upstream of both is not held fixed by the context. When the*

intersection is empty no unfixed node can reach both parents, and the two incoming paths are structurally unrelated.

Definition 4 (Diamond)**.** *A diamond is a triple $D = (R, C, \mathcal{E}_D)$. The relevant node set $R \subseteq V$ spans the reconvergent pattern. The conditioning set $C \subseteq V$ records which nodes identification fixed, or found already fixed, to isolate the pattern. The edge list $\mathcal{E}_D \subseteq E$ is induced on R by the membership rule constructed in Section 0.4.5 and characterised in Lemma 9.*

The module returns diamonds at two levels. A `DiamondsAtNode` record attaches to one join node its diamond together with the join's *non-diamond parents*, meaning the parents found unrelated to every reconvergent group at that join. The map `root_diamonds` collects these records for every join of G at the top level, and the diamonds recorded there are the network's *maximal diamonds*, identified at the network's own joins rather than inside another diamond. Separately, every diamond identified at any level, top-level or nested, is stored exactly once in `unique_diamonds`, keyed by the structural identity of Section 0.4.6. Each entry carries the diamond's fully analysed local structure (Section 0.6), and nested reconvergence appears as entries referencing other entries, so the store is a hierarchy rather than a flat list.

0.3 A Configurable Target Subgraph

The algorithms of this module are written against a definition of the special subgraph, not against the diamond in particular. Three predicates carry that definition. `is_det` decides when a node's reachability counts as already settled, `shared_fork` decides what counts as a correlating structure worth conditioning on, and `components` with infl decides when two parents count as independent. The identification and build algorithms do the same work whatever these predicates say. They find every subgraph of the input network whose structure satisfies the supplied properties, so a user can redefine the target around the network characteristics they care about and the module as it stands will decompose the network in those terms.

Apart from the `is_det` test, identification never reads the value map, and the structures it emits only carry the map restricted to their spans. The decomposition therefore behaves identically under all three of the framework's value types (deterministic, interval, and p-box), which is required of every module sitting between input processing and the analysis toolkits.

The diamond is the framework's default target, and as shipped the predicates encode the definitions of Section 0.2.

0.4 The Identification Algorithm

The identification procedure `new_identify` is the first of the module's two algorithms. It visits every join node of G once at the top level and recursively inside diamonds, and Algorithm 1 shows its recursive skeleton.

Algorithm 1 Recursive diamond identification (skeleton)

Require: join node v, context E, admissible parent set P
Ensure: list of `DiamondsAtNode` for v under E

```
partition P into groups by pairwise infl(·, E) intersection ▷ union–find; Section 0.4.2
for all groups G with |G| ≥ 2 do
    f ← covering candidate of G with minimum topi                ▷ Section 0.4.3
    construct diamond D for (v, G, f) under boundary B = E ∪ {f}  ▷ Section 0.4.5
    look up D in unique_diamonds by its identity key; reuse if present  ▷ Section 0.4.6
    for all join nodes j interior to D do
        recurse on (j, B, parents of j within D)                 ▷ residual reconvergence
    end for
end for
singleton groups become non-diamond parents of v
```

0.4.1 Conditioning only where correlation exists

By Definition 3, two parents of a join are correlated under E exactly when their unconditioned influence sets intersect. Extending the context with a node removes that node from every influence set, which is the factoring move of Section 0.1.1 applied inside a single network. The algorithm conditions only where an intersection actually exists and only on one node at a time. Nothing is enumerated exhaustively, and recursion stops as soon as the parents in view are pairwise unrelated.

0.4.2 Independence partitioning

At join v with admissible parents P and context E, the procedure `components`(P, E) evaluates $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E)$ for every pair and merges classes by union–find whenever the intersection is non-empty. The groups it returns are the connected components of the correlation relation (Theorem 6), and distinct groups share no unconditioned ancestry at all (Theorem 7), so each group can be decomposed on its own. Parents in singleton groups are related to nothing at v and become the join's non-diamond parents.

0.4.3 Fork selection

For a correlated group G, `shared_fork`(G, E) tallies every node that *covers* at least two members of the group. Coverage arises in two ways. A fork node may lie in the ancestry of

two or more members, which is the symmetric picture of a diamond. A member of G may instead be an ancestor of another member, the asymmetric case where the reconverging paths share no third node and the covering node is one of the parents itself. Both are genuine reconvergence and both are tested. Candidates already in E are skipped, since fixing an already fixed node does nothing, and so are candidates excluded by `is_det` (Section 0.4.7).

Among the surviving candidates the algorithm selects the one with minimum topological index, the covering node closest to the sources. Conditioning as far upstream as possible leaves the largest part of the group's shared structure intact downstream of the chosen node, where it stays available for reuse. Because topi is a strict total order the minimum is unique.

0.4.4 The recursive loop

Selecting fork f for group G at join v produces the diamond, constructed under the extended boundary $B = E \cup \{f\}$. It does not end the work at v, because conditioning on f need not resolve all correlation inside the pattern. Any interior join whose parents remain correlated under B gives rise to a nested diamond, found by the same procedure with the extended context. The recursion bottoms out when every group everywhere is a singleton, and Theorem 5 bounds its depth by $|V|$.

0.4.5 Edge construction and isolation

The diamond for group G at join v under boundary B spans

$$R \;=\; \{v\} \cup G \cup \bigcup_{p \in G} \mathrm{Anc}(p),$$

the join, the group, and everything upstream of the group, with induced edge list

$$\mathcal{E}_D \;=\; \big\{\, (u,w) \in E \;:\; u, w \in R,\;\; w \notin B,\;\; (w \neq v \text{ or } u \in G) \,\big\}.$$

The first two clauses confine the diamond to its span. The third cuts every edge into a boundary node, since a conditioned node's upstream structure is fixed and must not be re-entered, while its outgoing edges remain and feed the pattern as a local source. The fourth clause admits an edge into the join only from the group itself, and it exists to close a failure mode that the first three clauses cannot see.

Consider a nested level at which an enclosing diamond has already conditioned on a fork r, so $r \in E$, and suppose r is both an ancestor of some group member p and a direct parent of the join v. The partitioning of Section 0.4.2 handles r correctly. With $r \in E$ its influence set is empty, so it lands in a singleton group and its contribution to v

enters separately, fixed by the enclosing context. The span R is built differently, from the raw closure $\mathrm{Anc}(p)$, which contains r no matter what the context says. The partitioning works with context-filtered ancestry while the span works with raw ancestry, and the two silently diverge at every nested level, exactly where $E \neq \varnothing$. The edge (r, v) then has both endpoints in R and a head outside B, so the first three clauses admit it, and a diamond meant to capture v as reached through G alone quietly gains a second unrelated entry into its own join. The fourth clause blocks exactly this edge. Figure 1 shows the minimal instance, and the trace of Section 0.7.2 confirms that the implementation behaves as the clause requires, with the excluded parent surviving as a non-diamond parent and its direct edge to the join appearing in no diamond edge list.

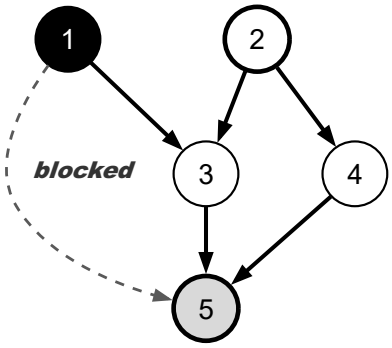

Figure 1: Edge isolation at join 5 in context $E = \{1\}$. Node 1 (filled) was conditioned by an enclosing diamond and is excluded from the group $G = \{3, 4\}$ with fork 2. Its direct edge $(1, 5)$, drawn dashed, is admitted by the first three filter clauses but blocked by the fourth. Bold edges form the nested diamond's edge list.

0.4.6 Context-aware identity

Every diamond is identified in `unique_diamonds` by the key

$$\mathrm{hkey}(D) \;=\; \big(\mathcal{E}_D,\; \{f\} \cup (E \cap R)\big),$$

its edge list together with its *relevant context*, meaning the fork it conditions on plus whatever already conditioned nodes lie in its span. The second term is what makes deduplication safe. The structural case that forces it involves a fork that is also a graph source. Such a node has no incoming edges, so the boundary cut of Section 0.4.5 does nothing to it, and the same edge list arises whether that fork is being conditioned at the current level or was already fixed one level up. The two situations are different objects, one a top-level diamond and the other nested inside the diamond that did the fixing, and a key of the form $(\mathcal{E}_D, \{f\})$ cannot tell them apart. Proposition 11 proves both that the

naive key genuinely collides and that the relevant-context key separates exactly the cases that must be separated.

0.4.7 The exclusion predicate `is_det`

One rule in identification consults declared node values. A node is excluded from conditioning candidacy when its reachability is already settled, which covers exactly two cases. A dead node, declared value 0, is never active, so nothing is reached through it. A certain source, declared value 1 with no incoming edges, is always reachable and has nothing upstream to resolve. The conjunction in the second case matters. A value-1 node that is not a source is still uncertain as a destination, because its reachability depends on its incoming edges, and it must remain conditionable. The simpler test of excluding every node valued 0 or 1 would disable every diamond in a network whose node values all happen to be 1, a configuration that real benchmark scenarios do use. For interval-valued and p-box-valued networks the predicate is conservatively false. It is one of the three override points of Section 0.3, and a user can replace it with an exclusion rule of their own.

0.5 Structural Correctness

All results in this section are statements about the structure of the input graph and nothing else. They divide along the line drawn in Section 0.3, with termination and determinism belonging to the recursive skeleton and grouping, disjointness, and completeness to the predicates that define the diamond.

Theorem 5 (Termination)**.** *The recursion of Algorithm 1 terminates after at most* $|V|$ *nested conditioning steps along any recursive path.*

Proof. Along any path of the recursion, each call either returns without recursing, which happens when every group at the join in view is a singleton, or selects a fork f and recurses with $B = E \cup \{f\}$. The candidate collection of Section 0.4.3 never offers a node already in E, so $f \notin E$ and $|V \setminus B| = |V \setminus E| - 1$ strictly. The quantity $|V \setminus E|$ is a non-negative integer that strictly decreases along the recursive path, and from its initial value $|V \setminus \varnothing| = |V|$ it admits at most $|V|$ decrements. □

Theorem 6 (Grouping correctness)**.** *Fix a join* v*, context* E*, and parent set* P*. Let* H *be the undirected graph on vertex set* P *with an edge between* p *and* q *if and only if* $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) \neq \varnothing$*. Then* `components`$(P, E)$ *returns exactly the connected components of* H*.*

Proof. The procedure evaluates the intersection test for every unordered pair in P, which by construction is precisely the edge relation of H, and performs a union–find merge

exactly on the pairs that pass. Union–find computes the partition of a vertex set into connected components given its edge relation. Applied to the vertices and edges of H, its output is the component partition of H. □

Theorem 7 (Group disjointness)**.** *If $G_i \neq G_j$ are distinct groups returned by* `components`(P, E)*, then*

$$\Big(\bigcup_{p \in G_i} \mathrm{infl}(p, E)\Big) \cap \Big(\bigcup_{q \in G_j} \mathrm{infl}(q, E)\Big) = \varnothing.$$

Proof. Let $p \in G_i$ and $q \in G_j$ be arbitrary. By Theorem 6 the two lie in different connected components of H, so (p, q) is not an edge of H. Since every pair was tested directly, the test itself must have failed, giving $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) = \varnothing$. The displayed intersection expands to the union of these pairwise intersections over all $p \in G_i$ and $q \in G_j$, and every one of them is empty. □

Disjointness is a statement about unconditioned influence sets, and it is what permits decomposing the groups separately. Whether separately computed values recombine correctly at the join is a different claim, which Chapter 5 makes and proves in its own terms.

Theorem 8 (Global completeness)**.** *Let v be a join node of G with parent set P, and suppose there exist distinct $p, q \in P$ and a node a with $\neg$*`is_det`(a) *such that either a is a fork node in $\mathrm{Anc}(p) \cap \mathrm{Anc}(q)$, or $p \in \mathrm{Anc}(q)$ (the asymmetric case, taking $a = p$). Then the top-level pass of* `new_identify` *stores a diamond for v in* `root_diamonds`*. Conversely, a join admitting no such pair receives no entry.*

Proof. At the top level $E = \varnothing$, so in either case $a \in \mathrm{infl}(p, \varnothing) \cap \mathrm{infl}(q, \varnothing)$ and the intersection is non-empty. By Theorem 6, p and q lie in a common group G with $|G| \geq 2$. In the candidate tally of Section 0.4.3, a is counted once for each member it covers, so from p and q alone its coverage is at least 2 and the candidate set is non-empty. A non-empty finite candidate set has a topi-minimal element, so `shared_fork` returns some fork, not necessarily a itself, Algorithm 1 constructs a diamond for G, and the top-level loop records it under v. For the converse, if no such pair exists then every pairwise influence intersection at v is empty, every group is a singleton, and the algorithm constructs nothing at v. That is the correct outcome, because the incoming paths of v are pairwise structurally unrelated. □

Lemma 9 (Edge isolation)**.** *Let D be constructed for group G at join v under boundary B as in Section 0.4.5. Then $(u, w) \in \mathcal{E}_D$ if and only if $(u, w) \in E$, $u, w \in R$, $w \notin B$, and ($w \neq v$ or $u \in G$). Consequently every directed path within $\mathcal{E}_D$ that terminates at v has its final edge originating in G, and no path within $\mathcal{E}_D$ passes through a node of B.*

Proof. The membership characterisation restates the filter defining $\mathcal{E}_D$. For the path properties, induct on path length. A length-one path into v is a single edge $(u, v) \in \mathcal{E}_D$, which the fourth clause forces to have $u \in G$. Any longer path has a final edge satisfying the same property, and each preceding edge (u, w) has $w \notin B$ and both endpoints in R by the first three clauses, so no intermediate node of the path lies in B and no edge of the path enters from outside R. □

Theorem 10 (Determinism)**.** *For a fixed input graph and iteration sets (G, L), join v, context E, and admissible parent set, the diamonds returned by Algorithm 1 are uniquely determined, independent of set or dictionary iteration order, thread scheduling, or any other accident of execution.*

Proof. By induction on recursion depth, it suffices that each step of one call is a function of its arguments alone. This holds for the partition step because, by Theorem 6, its output is the connected-component partition of H, a graph invariant that does not depend on the order in which pairs are examined or merges performed. It holds for the selection step because within each group the algorithm returns the minimum of a non-empty finite subset of V under topi, and a strict total order (Definition 1) admits exactly one minimum on such a set. Non-emptiness is guaranteed whenever the group has size at least two, by the tally argument of Theorem 8. The boundary $B = E \cup \{f\}$ and span R are then determined, hence so is $\mathcal{E}_D$, and hence so are the interior joins passed to the recursive calls, which are deterministic by the inductive hypothesis. □

The shape of this guarantee, a fixed total order pinning every choice point so that the output is a canonical function of the input, is also the shape of the canonicity guarantee of reduced ordered binary decision diagrams [11]. The mechanism is not the same. BDD canonicity depends on an externally supplied variable ordering, whereas topi is derived from a property of the input graph already proved in Chapter 3.

Proposition 11 (Context-aware identity: necessity and sufficiency)**.** *(i) There exist graphs, a source fork f, and contexts $E_1 = \varnothing$ and $E_2 \ni f$ under which the construction of Section 0.4.5 yields the same edge list $\mathcal{E}_D$, while the resulting diamonds belong at different levels of the nesting hierarchy. An identity of the form $(\mathcal{E}_D, \{f\})$ therefore conflates structurally distinct diamonds. (ii) Under* $\mathrm{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap R))$*, two constructions receive the same key only if they have the same edge list, condition on the same fork, and their contexts agree on every node of the span R. In that case they pose the identical sub-problem and deduplication is correct.*

Proof. (i) Let f be a fork that is also a graph source, and let v be a join reachable from f along two paths. If f is free and selected at the current level, so that $E_1 = \varnothing$, the boundary cut removes the edges into f, of which there are none, and produces some edge list $\mathcal{E}_D$. The diamond is top-level. If instead f was conditioned by an enclosing diamond,

so that $f \in E_2$, the same cut is applied for the same reason, again removes nothing, and produces the same $\mathcal{E}_D$. This diamond is nested inside the one that fixed f. The pair $(\mathcal{E}_D, \{f\})$ is identical in the two situations, but the diamonds are not interchangeable, since one must be recorded at the top level and the other inside its enclosing diamond's substructure.

(ii) Suppose $\text{hkey}(D_1) = \text{hkey}(D_2)$. Equality of edge lists forces equality of the spanned node sets R, and equality of $\{f\} \cup (E \cap R)$ then forces the two contexts to agree on every node of that shared span. Nodes outside R cannot affect the diamond's interior, because by Lemma 9 no path of $\mathcal{E}_D$ enters from outside R, so a node beyond the span has no route by which to influence the pattern whether it is conditioned or not. Two constructions that agree on the edge list, the fork, and the context restricted to the span therefore define the same sub-problem, and sharing one stored entry between them loses nothing. In the collision of part (i) the enclosing construction had conditioned at least one node lying in the span, namely f itself, so $E_1 \cap R \neq E_2 \cap R$ and the keys differ. The two diamonds that must be kept apart are kept apart. □

The key is also deliberately coarse in one direction. Constructions whose contexts differ only outside the span receive the same identity, and part (ii) is the reason this is reuse rather than a collision.

Taken together, these guarantees have a counting consequence. Consider a join whose k parents are all mutually correlated in the naive sense, with every parent sharing some ancestry with some other. Conditioning on the shared cutset jointly means enumerating its 2^k assignments. The decomposition replaces this in two moves, both already proved. The parent set is partitioned into groups whose influence sets are disjoint (Theorems 6 and 7), so the groups decompose independently. Each group, whatever its size, is then reduced to a single conditioning fork, one fork per group rather than one per correlated pair. In the extreme case where partitioning finds k separate groups, k constant-size single-fork problems stand where a 2^k enumeration stood.

The identification pass itself is polynomial. Per join it computes pairwise influence intersections under union–find and builds one span, and the `unique_diamonds` store bounds the number of distinct diamonds ever materialised. The exponential object in this problem is the downstream conditioning enumeration, and the decomposition's contribution is to make those enumerations as small and as widely reused as the network's structure allows.

0.6 Representation and Reuse

0.6.1 Diamonds as graph objects

Identification finds the diamonds. The module's second algorithm builds each one into a graph object of the same form as the full network. Each entry of `unique_diamonds` is a `DiamondComputationData` carrying the complete local structural analysis of one diamond, and its fields correspond one-for-one to the unified graph object of Chapter 3, as Table 1 shows.

Table 1: Field correspondence between the unified graph object of Chapter 3 and `DiamondComputationData`

Unified graph object	**Diamond entry**	**Role**
`outgoing_index`	`sub_outgoing_index`	Forward adjacency
`incoming_index`	`sub_incoming_index`	Reverse adjacency
`source_nodes`	`sub_sources`	Local sources of the span
`fork_nodes`	`sub_fork_nodes`	Local forks
`join_nodes`	`sub_join_nodes`	Local joins
`ancestors`	`sub_ancestors`	Closure restricted to the span
`descendants`	`sub_descendants`	Closure restricted to the span
`iteration_sets`	`sub_iteration_sets`	Layered order restricted to the span
`node_priors`	`sub_node_priors`	Value map restricted to the span
—	`sub_diamond_structures`	Nested diamonds interior to this one

Because a diamond satisfies the same object contract as the network it came from, anything the framework can do with the full network it can do with a diamond, including decomposing it further, which is what the recursion does. The analysis recorded in Table 1 is paid for once at identification time and reused on every later consultation.

The reductive use of a diamond noted in Section 0.1.1 rests on the same contract. A solved diamond can stand in the parent graph as a single equivalent node precisely because it satisfies that contract. Identification establishes only the compatibility, since performing the substitution needs a solved value, which structure alone cannot supply.

0.6.2 Deduplication by hash-consing

Before constructing a new entry, the algorithm looks the diamond's key up in `unique_diamonds`. A structurally identical diamond already resolved anywhere else in the recursion, under another maximal diamond or at another nesting level, is referenced rather than rebuilt. This is hash-consing in the standard sense [12, 13], a table keyed by structural identity so that equal structures are stored once and identity comparison becomes a key comparison.

The same technique appears as the unique table of a BDD package. Without the store the recursion is a tree and every call site is distinct. With it, structurally identical call sites collapse into shared entries and the representation becomes a DAG. The collapse loses nothing, because by Proposition 11(ii) two call sites receive the same key only when they pose the identical sub-problem.

0.7 Worked Examples

0.7.1 The atomic diamond

Figure 2 shows the minimal pattern, a source s feeding a fork f, two internally disjoint paths α and β, and a join j where they meet. At j both parents' influence sets contain f, so the parents form one group. Fork f is the only covering candidate, it is selected, and the diamond spans the whole figure with $C = \{f\}$.

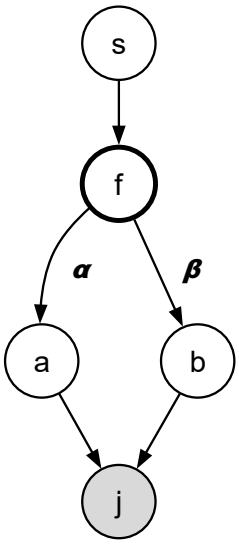

Figure 2: The atomic diamond.

0.7.2 A nested decomposition

The eight-node network of Figure 3 has forks 1 and 3 and joins 6 and 8. An outer reconvergence runs through fork 1 and meets at join 8, and on one of its branches an inner reconvergence runs through fork 3 and meets at join 6. The decomposition below is not hand-derived. It is the output of the implementation on this edge list, machine-checked, and the example of Figure 1 was verified in the same run.

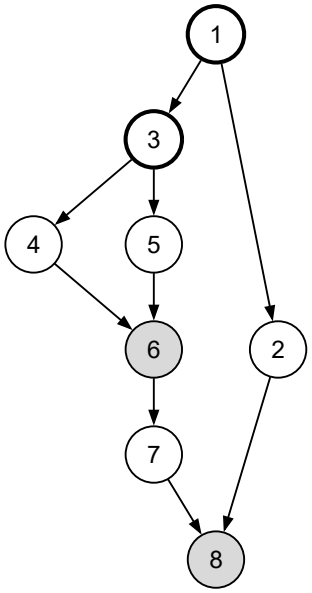

Figure 3: Frame 1. The network, with forks outlined in bold and joins shaded.

Join 8, top level. The parents are $\{2, 7\}$, with $\text{infl}(2, \varnothing) = \{1, 2\}$ and $\text{infl}(7, \varnothing) = \{1, 3, 4, 5, 6, 7\}$. The sets intersect in $\{1\}$, so the parents form one group. Fork 1 is the only candidate covering both and is selected, and the maximal diamond D_1 spans all nine edges with $C = \{1\}$ (Figure 4).

Join 6, inside D_1. Under the boundary $B = \{1\}$, interior join 6 has parents $\{4, 5\}$ with $\text{infl}(4, \{1\}) = \{3, 4\}$ and $\text{infl}(5, \{1\}) = \{3, 5\}$. The parents are still correlated through fork 3, so conditioning on 1 did not resolve the inner reconvergence, and this is the case the recursion exists for. The recursive call selects fork 3 and produces the inner diamond D_3 with $C = \{3\}$ and $\mathcal{E}_D = \{(3, 4), (3, 5), (4, 6), (5, 6)\}$. The edge $(1, 3)$ is cut by the boundary clause because its head is conditioned at this level (Figure 5).

Join 6, top level. Join 6 is also a join of the network itself, so the top-level pass visits it in its own right, and here the candidate set is larger. With $E = \varnothing$, forks 1 and 3 both cover parents 4 and 5, and the minimum-topi rule of Section 0.4.3 selects fork 1, the covering node closest to the source, rather than fork 3, which a reader tracing by eye might expect. The result is a second maximal diamond D_2 spanning $\{(1, 3), (3, 4), (3, 5), (4, 6), (5, 6)\}$ with $C = \{1\}$, and its interior recursion, now in context $\{1\}$, again produces exactly D_3.

The emitted hierarchy. In total the module returns two maximal diamonds and three unique diamonds (Figure 6). D_1 sits at join 8 and D_2 at join 6, each with $C = \{1\}$. The inner D_3 at join 6 with $C = \{3\}$ is stored once and referenced from the substructure tables of both D_1 and D_2 under the same key. Two different enclosing recursions posed the same sub-problem, with the same edge list, the same fork, and contexts agreeing on

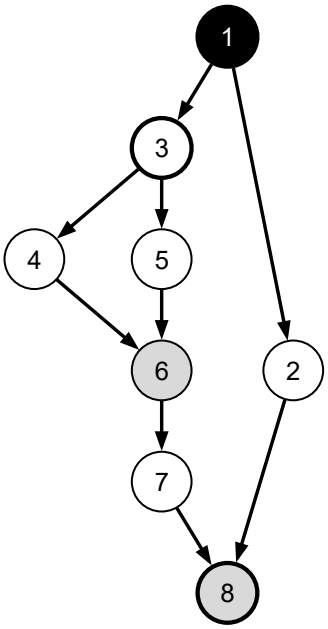

Figure 4: Frame 2. The maximal diamond at join 8, with fork 1 conditioned (filled) and the span covering the whole network.

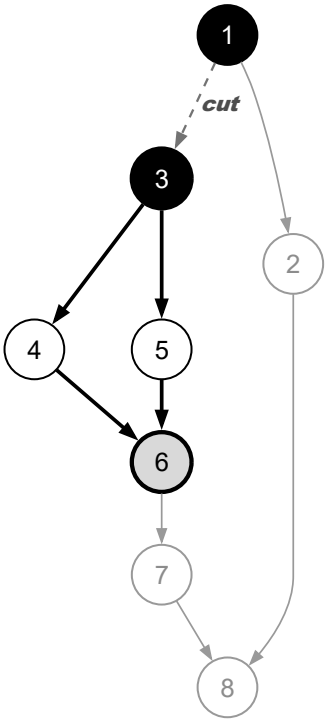

Figure 5: Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 selected, the inner diamond in bold, and edge $(1, 3)$ cut.

the span, and Proposition 11(ii) is the reason the single shared entry is correct.

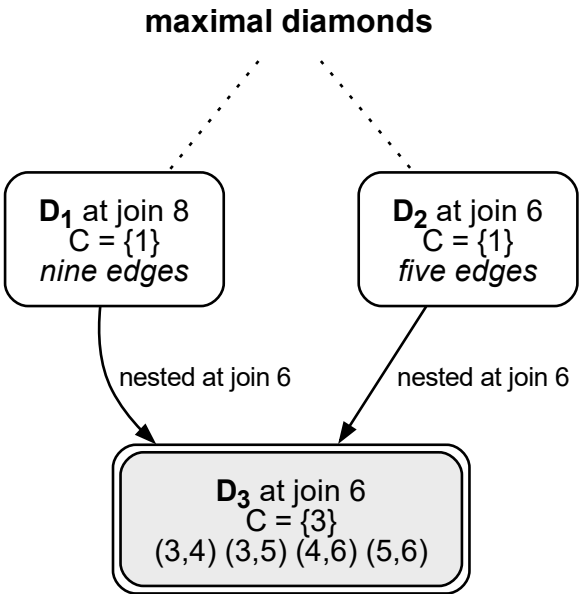

Figure 6: Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.

The trace also confirms the edge-isolation case of Section 0.4.5 end to end. In the network of Figure 1, the nested diamond at join 5 under $E = \{1\}$ is emitted with $C = \{2\} \cup (\{1\} \cap R) = \{1, 2\}$, which is the context-aware identity visible in the output. Node 1 is recorded as a non-diamond parent, and the edge $(1, 5)$ appears in no diamond edge list.

0.8 Summary

The module turns the graph object of Chapter 3 into two structures. `root_diamonds` records the network's maximal diamonds, and `unique_diamonds` stores every diamond found at any level exactly once, each entry a graph object of the same form as the network it came from. The decomposition terminates within $|V|$ nested steps and is canonical for a given input (Theorems 5 and 10). It groups a join's parents exactly by unconditioned shared ancestry and produces a diamond wherever reconvergence exists and only there (Theorems 6–8). Nothing outside a diamond's span enters its edge list (Lemma 9), and stored structure is shared exactly when sharing is correct (Proposition 11). At a join of k correlated parents the result is k single-fork conditioning problems in place of one 2^k enumeration.

Two questions are left open deliberately. One is how the values of independently decomposed groups recombine at a join. The other is the substitution of a solved diamond back into its parent graph as a single equivalent component. Both need values as well as structure, and the Probability Propagation Toolkit takes them up.

Bibliography

[1] M. Ouyang, "Review on modeling and simulation of interdependent critical infrastructure systems," *Reliability Engineering & System Safety*, vol. 121, pp. 43–60, 2014. DOI: 10.1016/j.ress.2013.06.040

[2] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, and S. Havlin, "Catastrophic cascade of failures in interdependent networks," *Nature*, vol. 464, no. 7291, pp. 1025–1028, 2010. DOI: 10.1038/nature08932

[3] H. Pérez-Rosés, "Sixty years of network reliability," *Mathematics in Computer Science*, vol. 12, pp. 275–293, 2018. DOI: 10.1007/s11786-018-0345-5

[4] A. Satyanarayana and R. K. Wood, "A linear-time algorithm for computing K-terminal reliability in series-parallel networks," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: 10.1137/0214057

[5] E. F. Moore and C. E. Shannon, "Reliable circuits using less reliable relays," *Journal of the Franklin Institute*, vol. 262, no. 3, pp. 191–208, 1956. DOI: 10.1016/0016-0032(56)90559-2

[6] A. Satyanarayana and M. K. Chang, "Network reliability and the factoring theorem," *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: 10.1002/net.3230130107

[7] L. B. Page and J. E. Perry, "A practical implementation of the factoring theorem for network reliability," *IEEE Transactions on Reliability*, vol. 37, no. 3, pp. 259–267, 1988. DOI: 10.1109/24.3752

[8] R. K. Wood, "A factoring algorithm using polygon-to-chain reductions for computing K-terminal network reliability," *Networks*, vol. 15, no. 2, pp. 173–190, 1985. DOI: 10.1002/net.3230150204

[9] N. Robertson and P. D. Seymour, "Graph minors. II. Algorithmic aspects of tree-width," *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. DOI: 10.1016/0196-6774(86)90023-4

[10] A. K. Goharshady and F. Mohammadi, "An efficient algorithm for computing network reliability in small treewidth," *Reliability Engineering & System Safety*, vol. 193, p. 106 665, 2020. DOI: 10.1016/j.ress.2019.106665

[11] R. E. Bryant, "Graph-based algorithms for Boolean function manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819

[12] A. P. Ershov, "On programming of arithmetic operations," *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, The origin of the hash-consing technique. DOI: 10.1145/368892.368907

[13] J.-C. Filliâtre and S. Conchon, "Type-safe modular hash-consing," in *Proceedings of the ACM SIGPLAN Workshop on ML*, 2006, pp. 12–19. DOI: 10.1145/1159876.1159880